ATLAS

SAP OPERATIONS SUITE

REPORTE TÉCNICO EJECUTIVO

Arquitectura y stack tecnológico de la plataforma ATLAS

Documento de referencia técnica sobre la solución de control y trazabilidad de RFCs (cambios) y solicitudes para la operación SAP. Describe arquitectura, tecnologías, persistencia, infraestructura, integraciones y procedimientos de despliegue y operación.

EAS Consulting × DXC Technology

Operación de servicios SAP

Versión de la aplicación

ATLAS v3.0

Fecha

23 de julio de 2026

00 Resumen y ficha técnica

ATLAS es una aplicación web para el registro, control y trazabilidad de RFCs (Requests for Change / ventanas de mantenimiento) y solicitudes operativas (KIT, Incidente, RCA, Actividad) del equipo de operación SAP. Ofrece dashboards ejecutivos, alertas de autorización (VOBO), gestión por roles y exportación hacia MS Project. La solución es **serverless**: un frontend estático servido desde GitHub Pages y un backend administrado (Backend-as-a-Service) sobre Supabase.

Tipo de solución	Aplicación web SPA (single-page), arquitectura serverless (estático + BaaS)
Frontend	Un solo archivo `index.html` (~6,250 líneas · 318 KB) — HTML5 + JavaScript + CSS, sin framework ni build
Backend	Supabase (PostgreSQL + Auth + API REST + Edge Functions serverless)
Hospedaje	Frontend en GitHub Pages · Backend en Supabase Cloud (región us-west-1)
URL de producción	`antoniolopezlabra-boop.github.io/tqs-rfc-manager`
Autenticación	Supabase Auth (JWT) con control por roles y seguridad a nivel de fila (RLS)
Roles	admin · tqs_leader · fte · director

01 Arquitectura de la solución

ATLAS sigue un patrón **cliente-servidor serverless**: toda la lógica de presentación y negocio de la interfaz reside en el navegador (cliente pesado), consumiendo directamente los servicios administrados de Supabase mediante HTTPS. No existe un servidor de aplicación propio; las operaciones privilegiadas (p. ej. alta/baja de usuarios) se resuelven en **funciones serverless (Edge Functions)** puntuales.

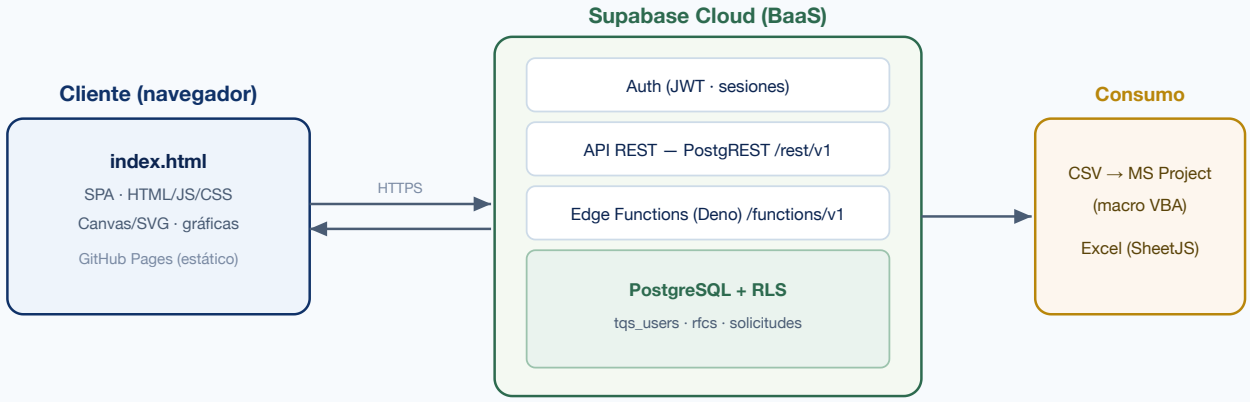


Figura 1. Arquitectura lógica de ATLAS: cliente estático + backend administrado + salidas de exportación.

Características de diseño: **sin paso de build** (el HTML se sirve tal cual), **sin infraestructura propia** (cómputo y base de datos administrados), y **autorización delegada al servidor** mediante Supabase Auth + políticas RLS de PostgreSQL, con funciones serverless para las operaciones que requieren privilegios elevados.

02 Lenguajes de programación y frameworks

Capa	Lenguajes / tecnologías
Interfaz (frontend)	HTML5, CSS3 (hojas de estilo propias, **sin framework CSS**), JavaScript ES2020 **vanilla** (sin framework SPA — no React/Vue/Angular). Gráficas con **Canvas 2D API** y **SVG** (sin librería de charts).
Lógica serverless	**TypeScript** sobre **Deno** (Supabase Edge Functions).
Base de datos	**SQL** (PostgreSQL); PL/pgSQL para triggers y políticas RLS.
Build / tooling	Ninguno — no hay transpilación ni empaquetado; despliegue directo del archivo fuente.

> Decisión de arquitectura: mantener la aplicación en un único archivo autocontenido reduce la complejidad operativa (sin cadena de build, sin node_modules en producción) a costa de un archivo fuente grande. Las dependencias se cargan por CDN.

03 Componentes, librerías y dependencias principales

Dependencia	Versión	Uso
@supabase/supabase-js	v2 (UMD)	Cliente oficial de Supabase: autenticación (login JWT) y acceso a datos.
SheetJS (xlsx)	0.20.3	Importación/exportación Excel: carga masiva de RFCs y descarga de plantilla.
Google Fonts	—	Tipografías *Exo 2* (títulos) e *Inter* (texto).
APIs nativas del navegador	—	Canvas 2D y SVG (gráficas), Fetch (REST), localStorage (sesión y preferencias).

Distribución y seguridad de dependencias

- Las librerías se cargan por **CDN** (jsDelivr / SheetJS CDN); no hay `node_modules` ni gestor de paquetes en el frontend.
- La página declara una **Content-Security-Policy (CSP)** que restringe los orígenes permitidos para scripts, estilos y fuentes (jsDelivr, SheetJS CDN, Google Fonts).
- Backend serverless: la Edge Function usa `@supabase/supabase-js` vía `esm.sh` en tiempo de ejecución Deno.

04 Base de datos y modelo de persistencia

Persistencia principal en **PostgreSQL administrado por Supabase**. El cliente accede vía API REST autogenerada (PostgREST); las operaciones privilegiadas pasan por Edge Functions con clave `service_role`.

Tabla	Columnas	RLS	Contenido
tqs_users	10	4 políticas	Usuarios / perfiles y su rol; vinculada a Supabase Auth.
rfcs	20	4 políticas	RFCs / ventanas de mantenimiento (registro central).
solicitudes	13	4 políticas	Solicitudes operativas: KIT, Incidente, RCA, Actividad (con bitácora y action items).
profiles · sistemas	5 · 7	—	Tablas de la versión 1 (heredadas), actualmente en desuso.

- **Autenticación:** Supabase Auth (esquema `auth.users`) emitiendo JWT; el perfil y rol viven en `tqs_users`.
- **Seguridad a nivel de fila (RLS):** activa en las tres tablas operativas (4 políticas cada una), aplicando la autorización en el servidor según identidad y rol.
- **Persistencia local:** `localStorage` para la sesión y preferencias de la interfaz (p. ej. tema).

05 Infraestructura y ambiente de despliegue

Componente	Detalle
Frontend (hosting)	**GitHub Pages** — hosting estático con HTTPS y CDN de GitHub. Repositorio `github.com/antoniolopezlabra-boop/tqs-rfc-manager` (público), rama `main`, despliegue automático al hacer push.
Backend	**Supabase Cloud** — PostgreSQL, Auth, API REST (PostgREST) y Edge Functions (Deno). Organización “TQS - EAS” (plan gratuito).
Región	EE. UU. Oeste — `us-west-1` (Norte de California).
URL de producción	`https://antoniolopezlabra-boop.github.io/tqs-rfc-manager/`
Servidores propios	Ninguno — arquitectura totalmente serverless y estática (sin VMs ni contenedores que administrar).

> Consideración: el repositorio del frontend es público. El acceso a los datos está protegido por Supabase Auth + RLS del lado del servidor; ninguna credencial privilegiada (service_role) reside en el cliente.

06 Integraciones con otros sistemas o servicios

Sistema / servicio	Tipo de integración
Supabase	Núcleo del backend: Auth, API REST y Edge Functions (consumo directo por HTTPS).
Microsoft Project	Exportación de RFCs a **CSV con formato y prioridad MPP**, ingerido por una macro VBA en MS Project para actualizar el plan.
Microsoft Excel	Carga masiva de RFCs y descarga de plantilla mediante SheetJS.
Google Fonts	Provisión de tipografías.

Histórico y hoja de ruta

- **Histórico (v1):** el backend original usaba **Google Sheets + Google Apps Script**; fue reemplazado por Supabase.
- **Planeado:** extracción asistida de RFCs desde correo con IA y notificaciones por WhatsApp.

07 Procedimiento de despliegue y operación

Despliegue

Elemento	Procedimiento
Frontend	Editar `index.html` → `git push` a la rama `main` → GitHub Pages reconstruye y publica automáticamente (≈1–2 min). Recomendado recargar con caché limpia (hard refresh).
Edge Functions	Despliegue con Supabase CLI: `supabase functions deploy admin-users`.
Base de datos	Migraciones SQL aplicadas vía Supabase (consola web, CLI o Management API).

Operación — roles

Rol	Permisos
admin	Superusuario: lectura/escritura total y gestión de usuarios y sistemas.
tqs_leader	Registra y edita sus propios RFCs; lee todos.
fte	Solo lectura de todos los sistemas.
director	Solo lectura, con acceso al Panel Ejecutivo y dashboards de dirección.

Flujo operativo

Inicio de sesión (Supabase Auth) → registro de RFCs y solicitudes → seguimiento en dashboards y alertas de VOBO (autorización) → exportación a CSV para MS Project. Los cambios en código quedan versionados en Git con historial descriptivo.

08 Documentación técnica disponible

Documento	Descripción
README.md	Descripción general: stack, roles, sistemas SAP y flujo de actualización (en el repositorio).
schema.sql	Definición del esquema de base de datos (línea base) — tablas, triggers y políticas.
supabase/config.toml	Configuración del proyecto Supabase (CLI / entornos).
supabase/functions/…/index.ts	Código fuente de la Edge Function `admin-users` (gestión de usuarios del lado servidor).
Historial Git	Bitácora de cambios versionada con mensajes de commit descriptivos.
Este documento	Reporte Técnico Ejecutivo (arquitectura y stack).

> Nota: `schema.sql` refleja la línea base de la versión 1; el esquema en producción ha evolucionado (Supabase Auth, RLS, tabla `solicitudes`). Para el estado vigente, la fuente de verdad es la base de datos de Supabase.